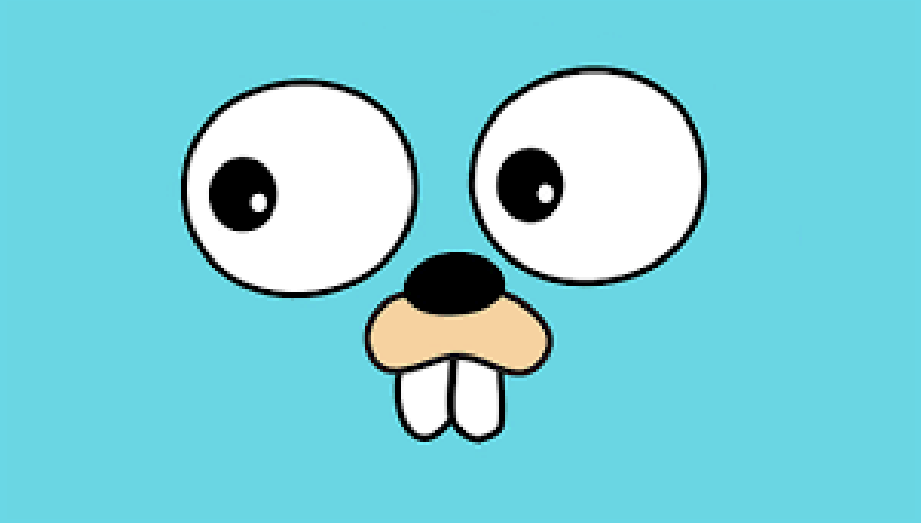

Package Management and Modules

Discussion Points

- Creating and using packages
- Understanding Go modules
- Versioning and dependency management with go.mod

Creating and using packages

Introduction to Packages?

A package in Go is essentially a named collection of one or more related .go files. In Go, the primary purpose of packages is to help you isolate and reuse code.

Good package names are short and clear. They are lower case, with no under_scores or mixedCaps. They are often simple nouns, such as:

- time (provides functionality for measuring and displaying time)
- list (implements a doubly linked list)
- http (provides HTTP client and server implementations)

The style of names typical of another language might not be idiomatic in a Go program. Here are two examples of names that might be good style in other languages but do not fit well in Go:

- computeServiceClient
- priority_queue

Creating and using packages

Main Package?

In Go, main is actually a special package name which indicates that the package contains the code for an executable application. That is, it indicates that the package contains code that can be built into a binary and run.

Any package with the name main MUST also contain a main() function somewhere in the package which acts as the entry point for the program

It's conventional for your *main()* function to live in a file with the filename **main.go**. Technically it doesn't have to, but following this convention makes the application entry point easier to find for anyone reading your code in the future.

Creating and using packages

Importing and using standard library Packages?

Individual .go files can import and use exported types, constants, variables and functions from other packages — including the packages in the Go standard library.

When importing a package from the standard library you need to use the full path to the package in the standard library tree, not just the name of the package:

```
package main

import (
    "fmt"
    "math/rand"            // Not "rand"
    "net/http"             // Not "http"
    "net/http/httptest" // Not "httptest"
)
```

Creating and using packages

Init() in Packages?

Init() functions can be used within a package block and regardless of how many times that package is imported, the init function will only be called once

```
package main

func init() {
    println("This will be called without explicit call")
}

func main() {
    println("Main function started")
}

//This will be called without explicit call
//Main function started
```

Additionally, you can have multiple init() functions per package; they will be executed in the order they show up in the file (after all variables are initialized of course). If they span multiple files, they will be executed in lexical file name order

It's incredibly important to note that you cannot rely upon the order of execution of your init() functions. It's instead better to focus on writing your systems in such a way that the order does not matter.

Understanding Go modules

What is Modules?

If you have a small application which only imports packages from the standard library, then what we've done so far works just fine. But if you want to import and use a third-party package — or structure your code, so it's split into multiple packages — then you first need to turn your code into a Go module

A module is... a tree of Go source files with a go.mod file in the tree's root directory.

```
go mod init github.com/zaur-adigozalov/go-pkg
```

In the Go community it's conventional to base your module path on a URL that you own or control.

What is Modules?

It's not necessary for a main package to live in the module root. It can be anywhere. In fact, it's totally OK for a module to contain multiple main packages. For example, in a larger project you could have a *cmd/cli* directory with the main package for a CLI tool, and a *cmd/web* directory with the main package for a web application in the same module.

Non-main packages don't need to be a direct child of the module root either. They can be anywhere in an arbitrarily deep directory structure within the module.

*The directory name **internal** has a special behavior in Go. Any packages which live under a directory called internal can only be imported by code inside the parent of the internal directory.*

Importing and using 3rd-party packages?

To use third-party packages first you need to download the third-party code from its public repository to your local machine, with **go get**:

go get github.com/jessevdk/go-flags

One more interesting and useful command is **go mod tidy** which ensures that the go.mod file matches the source code in the module.

To update go dependencies

go get –u ./…

It adds any missing module requirements necessary to build the current module's packages and dependencies, and it removes requirements on modules that don't provide any relevant packages. It also adds any missing entries to go.sum and removes unnecessary entries.

Internal package in Go?

The core of using internal packages lies in their ability to manage visibility and establish clear boundaries within a Go program. By using internal packages, developers can ensure that certain parts of their codebase are "shielded" from external use, thus maintaining a clean and well-defined interface for users and other parts of the program.

To take advantage of this feature, you simply need to place the desired package within a directory named **internal**, or within any subdirectory of an **internal** directory. The Go toolchain enforces access restrictions based on this convention.

Thank you.
Questions?

“Learn from yesterday, live for today, hope for tomorrow. The important thing is not to stop questioning.”

- Albert Einstein